

MODULE 06

Containerization with Docker

Day 2 · Core MVP Module

Containerization with Docker



SDK: Build Stage

multi-stage copy



Runtime: Final Image

LEARNING OBJECTIVES

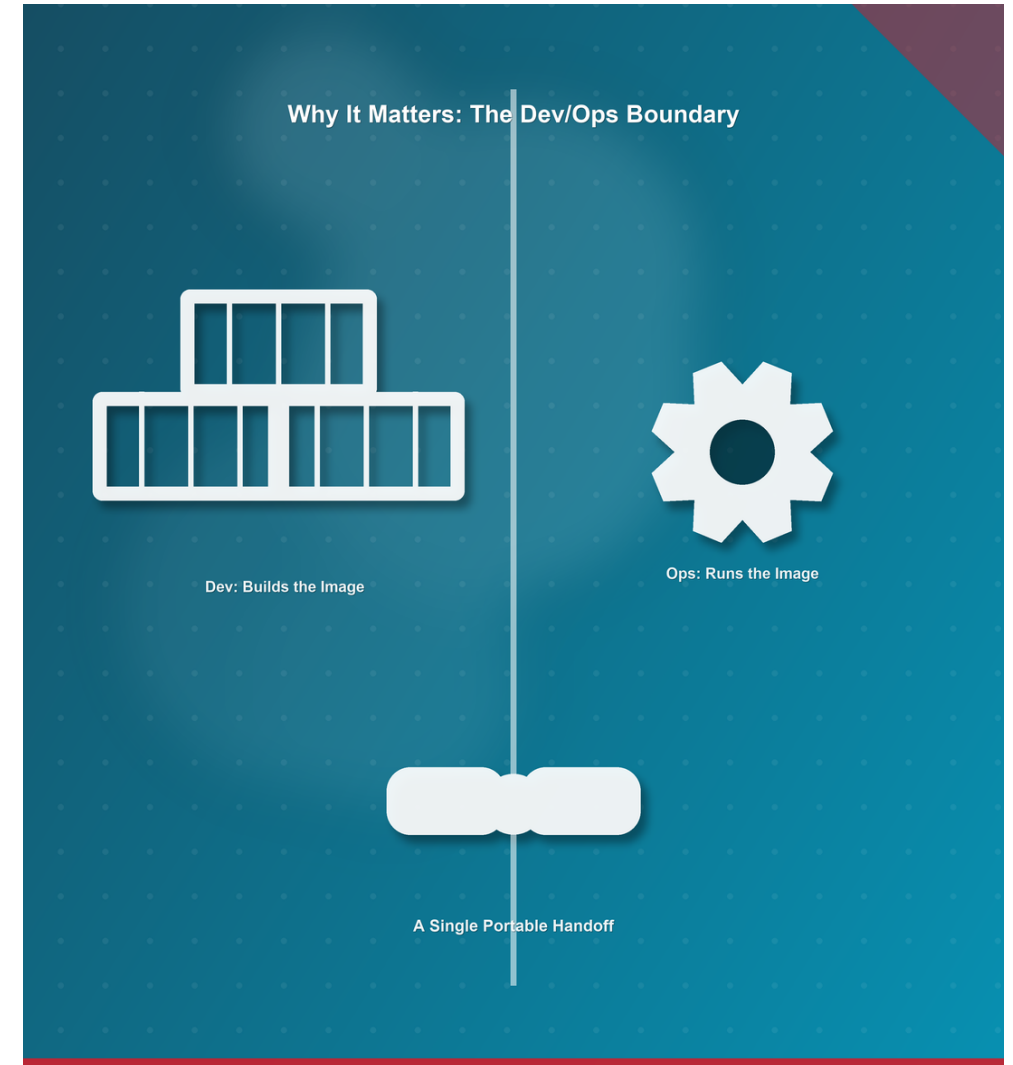
- Build and run .NET application images using multi-stage Dockerfiles.
- Validate health and runtime behavior in a containerized environment.
- Prepare images for registry and pipeline use.



WHY THIS MODULE MATTERS

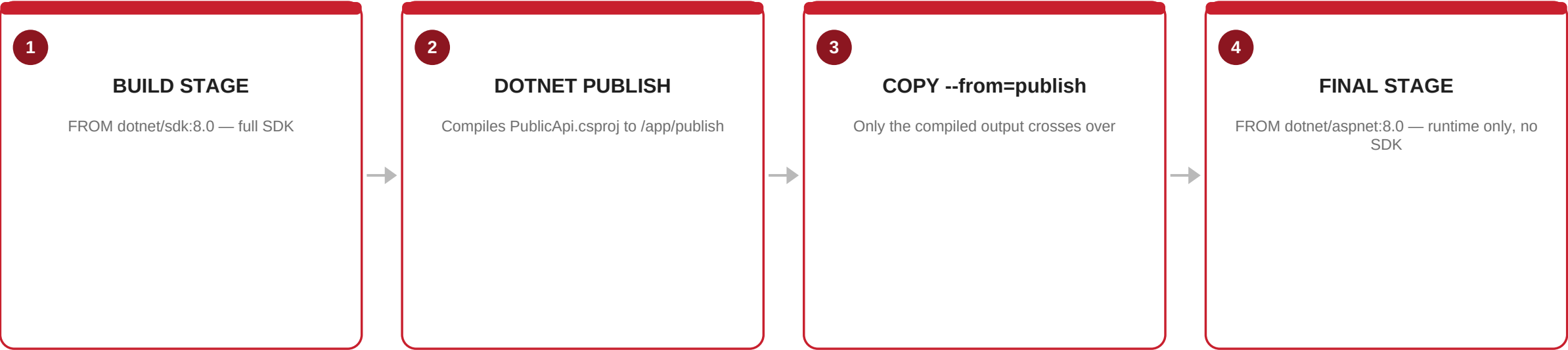
Containerization is the handoff boundary between development and platform operations.

This module establishes that boundary using repeatable image build practices.



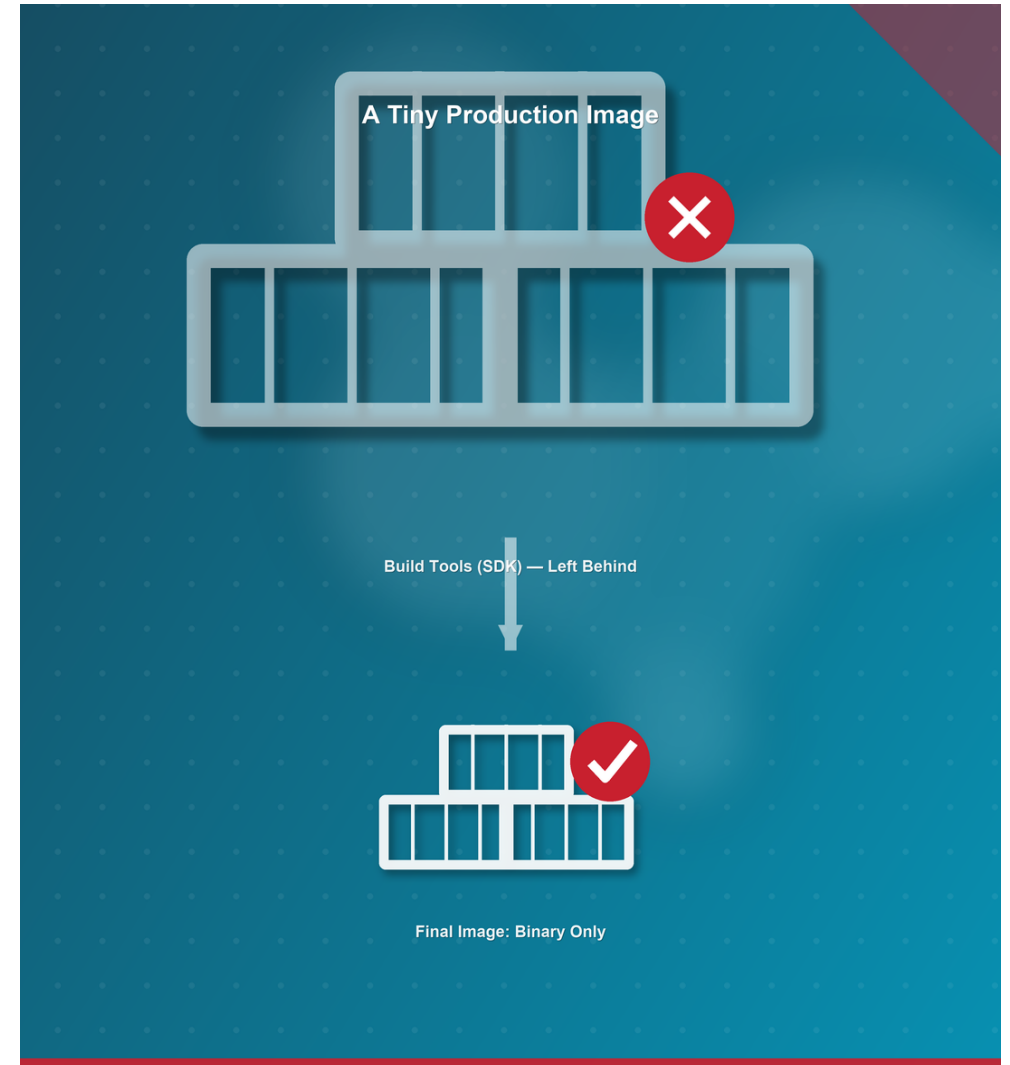
CONCEPT MAP

The exact two-stage build the eShopOnWeb Dockerfile uses to keep the shipped image small.



FROM THE DOCS: WHY MULTI-STAGE BUILDS

- Docker's own goal for multi-stage builds: optimize image size while keeping Dockerfiles easy to read and maintain.
- Multi-stage builds selectively copy artifacts between stages, leaving build tools out of the final image.
- Docker's own description of the result: “a tiny production image with nothing but the binary inside.”
- Name a stage with `FROM <image> AS <name>`; pull from it later with `COPY --from=<name>`.
- `COPY --from` is reorder-safe — stages can be reordered later without breaking the copy.



COURSEWARE FOCUS

- Multi-stage builds
- Image management
- Local orchestration



REAL MULTI-STAGE DOCKERFILE — ESHOPONWEB

REAL CODE — src/PublicApi/Dockerfile

```
FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS base
WORKDIR /app
EXPOSE 80
EXPOSE 443

FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
WORKDIR /app
COPY . .
WORKDIR "/app/src/PublicApi"
RUN dotnet restore
RUN dotnet build "./PublicApi.csproj" -c Release -o /app/build

FROM build AS publish
RUN dotnet publish "./PublicApi.csproj" -c Release -o /app/publish

FROM base AS final
WORKDIR /app
COPY --from=publish /app/publish .
ENTRYPOINT ["dotnet", "PublicApi.dll"]
```

Same FROM...AS / COPY --from pattern the Docker docs describe, in the app you'll containerize in Lab 06.

TOOLS & RESOURCES

Official Repo

dotnet/dotnet-docker (Official Microsoft .NET Docker Samples)

Multi-stage Dockerfile templates for ASP.NET Core (Linux/Windows).

Microsoft Learn

Build and run a containerized web app with Docker and Azure Container Registry

ACR registry setup, image building, tagging, and pushing.

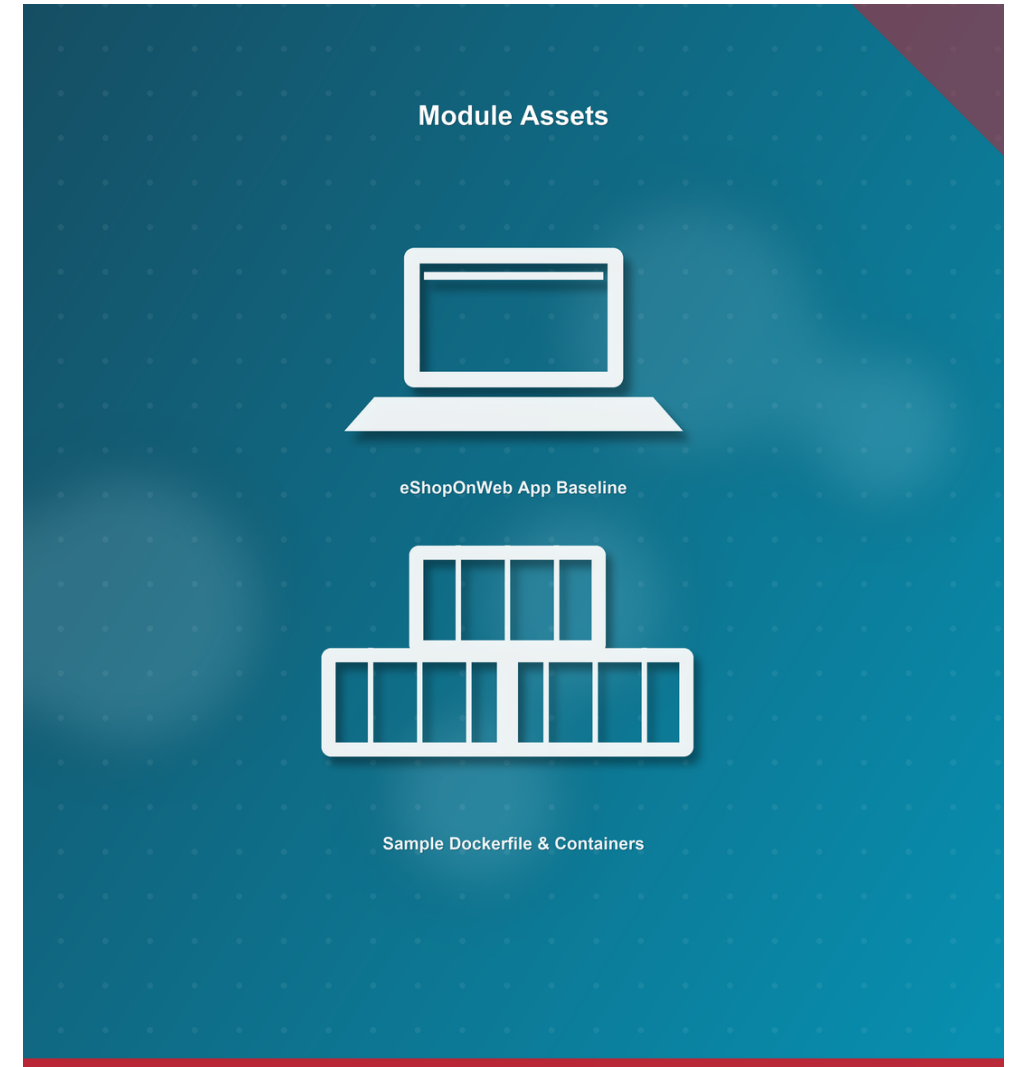
Local Tooling

Docker Desktop / Podman CLI & Docker Compose

Multi-container local orchestration (App + SQL Server).

SUPPORTING ASSETS

- App baseline: `course/repos/eShopOnWeb`
- Container examples: `course/repos/samples`



LAB 06: CONTAINERIZATION WITH DOCKER

Lab 06: Containerization with Docker



Build the Image



Run & Verify

LAB STEPS

Goal: Containerize the app and verify local runtime behavior.

Tools: Docker or Podman, .NET SDK · Cloud: optional Azure Container Registry.

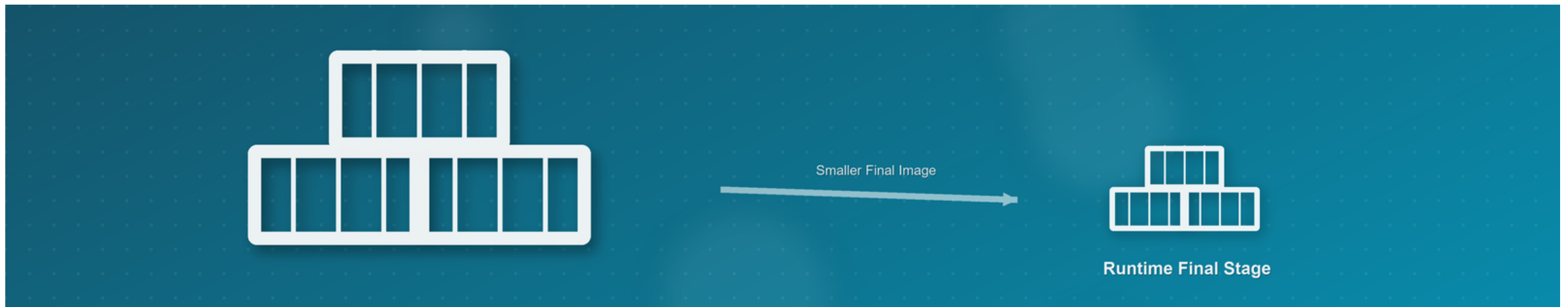
- Step 1 — Create or update a multi-stage Dockerfile.
- Step 2 — Build the image locally.
- Step 3 — Run the container and test the health endpoint.
- Step 4 — Add or verify compose configuration if needed.
- Step 5 — Tag the image for pipeline or registry use.
- Validation: the image builds successfully and the container responds to health checks.



KNOWLEDGE CHECK: Containerization with Docker

What is the main benefit of a multi-stage Dockerfile for a .NET application?

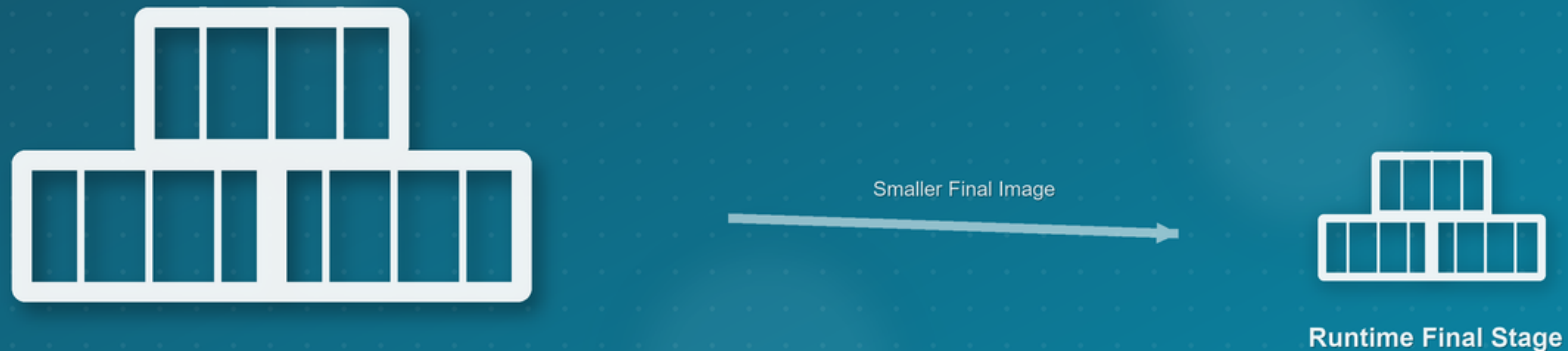
- A. It removes the need for a runtime environment
- B. It builds the app in one stage and copies only the compiled output into a smaller final runtime image
- C. It automatically deploys the app to Kubernetes
- D. It replaces the need for a database



KNOWLEDGE CHECK: Containerization with Docker

What is the main benefit of a multi-stage Dockerfile for a .NET application?

- A. It removes the need for a runtime environment
- B. It builds the app in one stage and copies only the compiled output into a smaller final runtime image**
- C. It automatically deploys the app to Kubernetes
- D. It replaces the need for a database



KNOWLEDGE CHECK: Containerization with Docker

In the eShopOnWeb Dockerfile, why does the final image use the aspnet:8.0 base instead of the sdk:8.0 image used to build it?

- A. aspnet is only faster to download
- B. The final stage only needs the runtime, not the full SDK, keeping the image smaller
- C. sdk images can't run in containers
- D. There is no difference



aspnet:8.0 (runtime, small)



sdk:8.0 (build tools, large)

KNOWLEDGE CHECK: Containerization with Docker

In the eShopOnWeb Dockerfile, why does the final image use the aspnet:8.0 base instead of the sdk:8.0 image used to build it?

- A. aspnet is only faster to download
- B. The final stage only needs the runtime, not the full SDK, keeping the image smaller**
- C. sdk images can't run in containers
- D. There is no difference



aspnet:8.0 (runtime, small)



sdk:8.0 (build tools, large)

SOURCES & FURTHER READING

EXTERNAL DOCUMENTATION

■ Multi-stage builds — Docker Docs

<https://docs.docker.com/build/building/multi-stage/>

REFERENCED IN THIS REPOSITORY

■ `course/repos/eShopOnWeb/src/PublicApi/Dockerfile`

■ `course/repos/eShopOnWeb/docker-compose.yml`

RECAP & SUCCESS CRITERIA

- Containerized app starts and serves expected responses.
- Tier: Core MVP Module

